



Unidad VII

Clase 13

Grafos

RESOLVER PROBLEMAS CON GRAFOS

Aplicar grafos a problemas integrales con estrategia y modularidad.

Resolver problemas con grafos: de la red a la estrategia

El Cambio de Paradigma

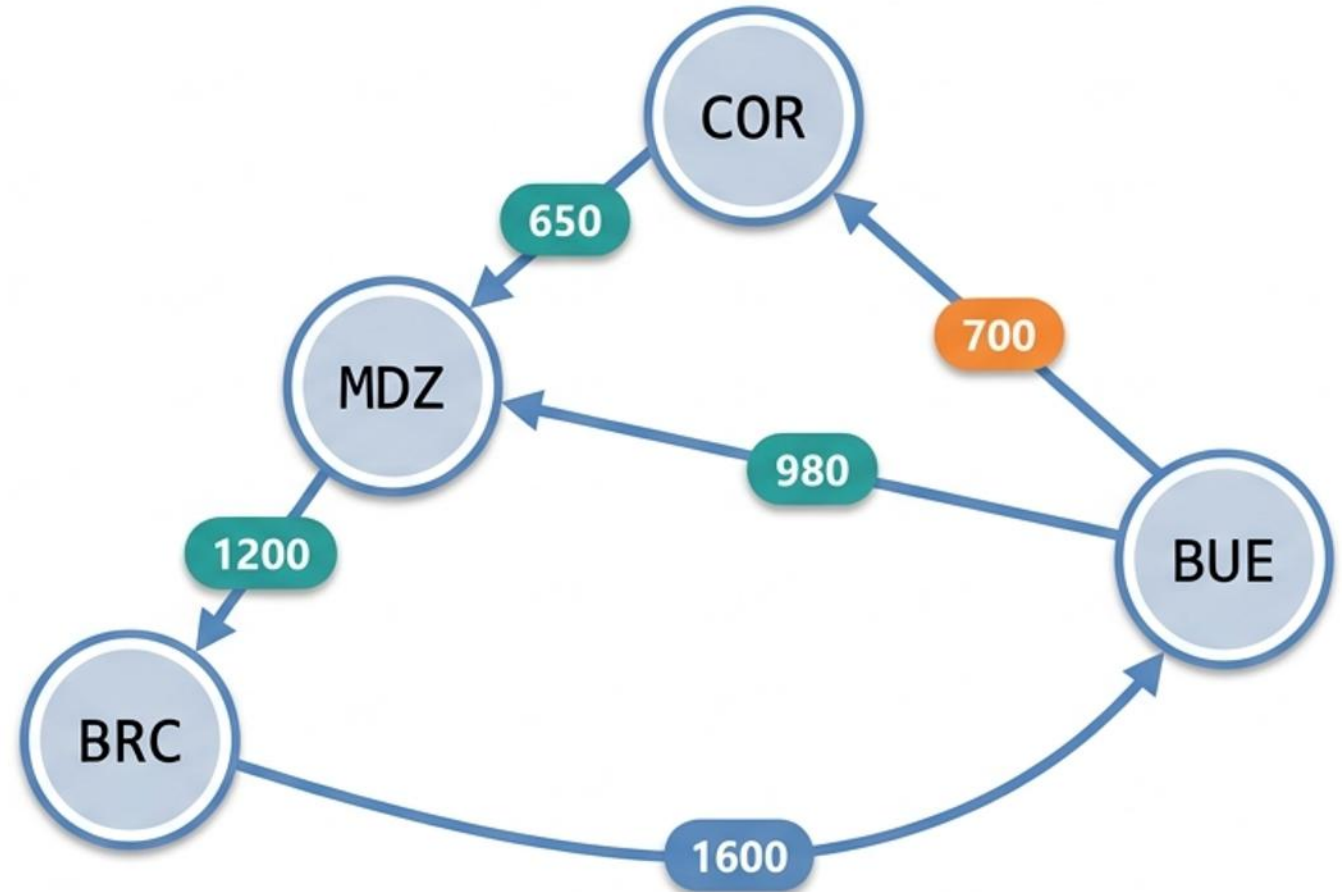
A diferencia de las estructuras jerárquicas (árboles), los grafos modelan relaciones complejas y multidireccionales.

El Propósito

Transformar realidades abstractas (logística, dependencias, vuelos) en redes computables para responder preguntas estratégicas.

Más Allá de la Existencia

Modelan conexión, alcance, caminos intermedios y rutas convenientes.



No alcanza con saber que los nodos existen; importa saber cómo se conectan, si se puede llegar y cuánto cuesta llegar.

Grafo dirigido y ponderado: qué significa cada decisión

Vértice y Arista

El **vértice** representa la **entidad** (ciudad, usuario, proceso); la **arista** representa la **conexión lógica** entre ellos.



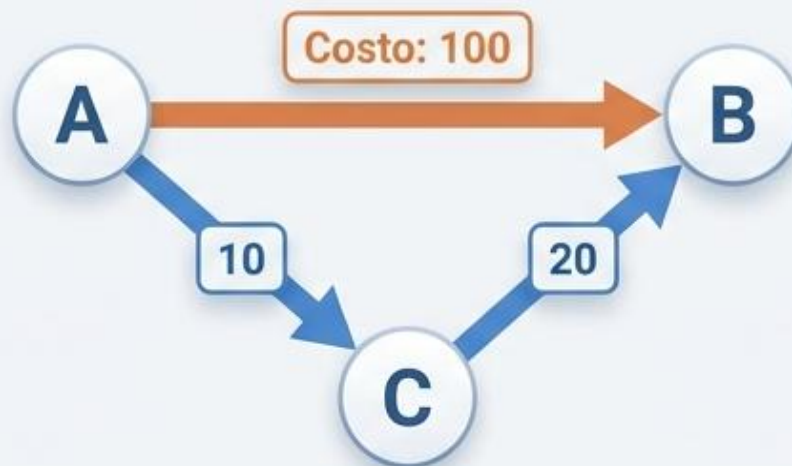
Dirección



Una relación **dirigida** no es **bidireccional** por defecto. Una arista A -> B es distinta de B -> A.

Peso (Weight)

La arista almacena un **valor métrico fundamental**. Representa costo, distancia, tiempo, riesgo o prioridad.



El camino directo (100) es menos conveniente que el indirecto (30).

La Decisión



La **ponderación** permite a los **algoritmos de optimización** descartar rutas directas ineficientes a favor de caminos indirectos más rentables.

Operaciones primitivas del GrafoTDA

El **Contrato de Abstracción**: El TDA define qué podemos hacer con la estructura sin exponer cómo está implementada internamente.

Construcción

```
InicializarGrafo()  
AgregarVertice(v)  
AgregarArista(v1, v2, peso)
```

Modificación

```
EliminarVertice(v)  
EliminarArista(v1, v2)
```

Consulta

```
Vertices()  
ExisteArista(v1, v2)  
PesoArista(v1, v2)
```

Consultas Primitivas (Mini-Explainer)

```
ExisteArista(10, 20)
```

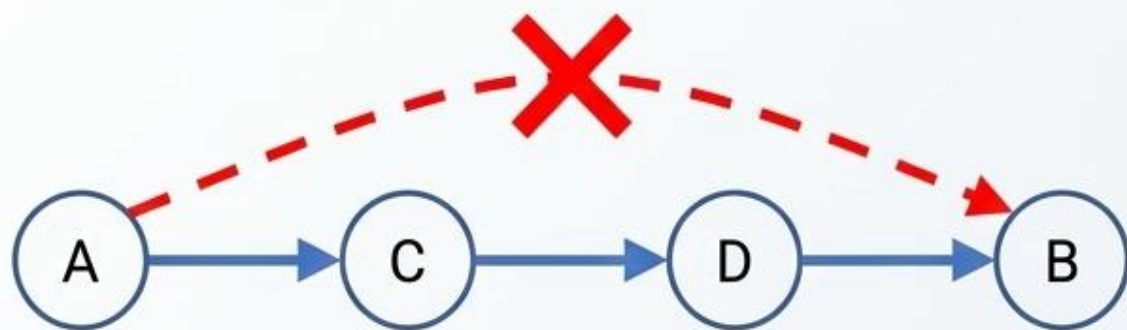
Responde una pregunta booleana estricta: ¿Hay una conexión directa desde el origen 10 hacia el destino 20?

```
PesoArista(10, 20)
```

Recupera la métrica (el costo o distancia) asociada exclusivamente a esa conexión directa.

Operaciones primitivas vs algoritmos derivados

Diferenciar claramente una consulta a la estructura base ($O(1)$ o muy rápida) de una pregunta topológica que requiere recorrer el grafo sistemáticamente.



`ExisteArista(A, B) == FALSE`

Pero **B** sí es alcanzable indirectamente.

Pregunta de Negocio / Lógica	Tipo de Resolución	Herramienta Computacional
Conexión directa: ¿Hay arista de A hacia B?	Consulta Primitiva	ExisteArista(A, B)
Alcanzabilidad: ¿Puedo llegar de A a B?	Algoritmo de Recorrido	BFS o DFS
Menor cantidad de saltos: ¿Ruta con menos escalas?	Algoritmo de Distancia	BFS (por niveles)
Menor costo: ¿Ruta más barata/rápida?	Algoritmo de Optimización	Dijkstra

Matriz o lista de adyacencia: impacto en implementación

Matriz de Adyacencia

	0	1	2	3	4
0	0	15	0	0	0
1	0	0	0	42	0
2	0	0	0	0	7
3	88	0	0	0	0
4	0	0	33	0	0

- **Mecánica:** Consumo de memoria cuadrático ($N \times N$).
- **Ventaja:** Consultas directas instantáneas (ExisteArista en $O(1)$).
- **Desventaja:** Gran desperdicio de memoria si existen pocas conexiones.
- **Criterio Práctico:** Usar para grafos densos o donde la validación de conexiones directas es la operación crítica.

Lista de Adyacencia

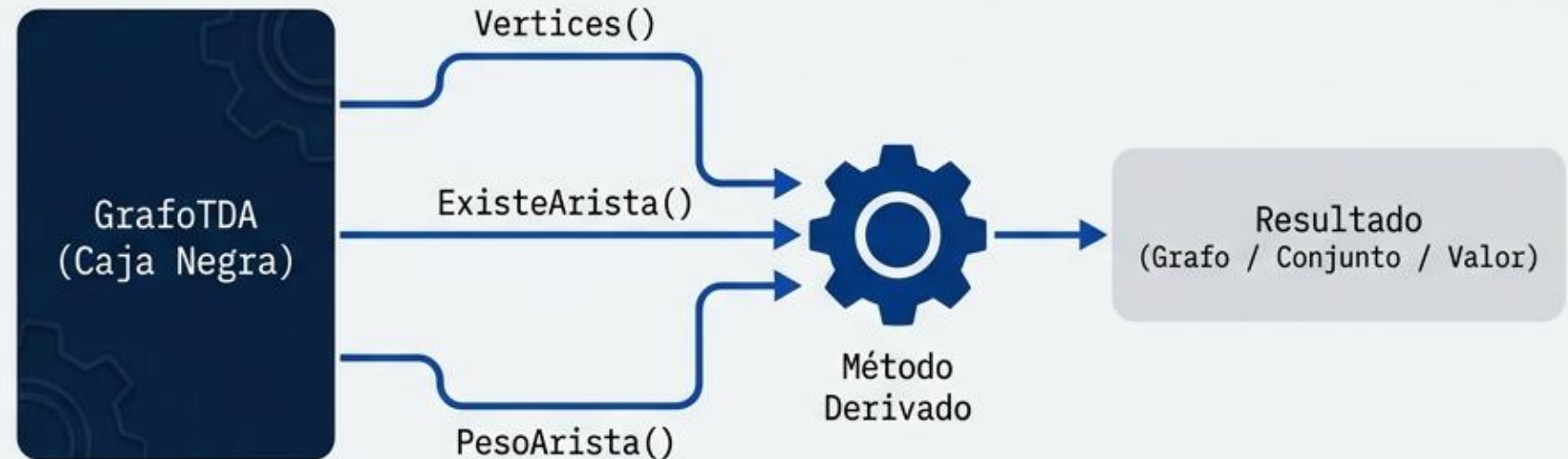


- **Mecánica:** Arreglo de vértices con listas enlazadas de vecinos.
- **Ventaja:** Altamente eficiente en memoria; facilita recorrer el vecindario inmediato.
- **Desventaja:** Verificar una arista puntual requiere iterar la sub-lista.
- **Criterio Práctico:** Usar para grafos dispersos y cuando la prioridad sea ejecutar algoritmos de recorrido (BFS/DFS).

Construir soluciones sobre el GrafoTDA

La potencia de un grafo no reside en su estructura interna (Matriz o Lista), sino en cómo combinamos sus operaciones primitivas.

- **El Principio de la Caja Negra:** El algoritmo no accede directamente a la memoria ni a los arreglos.
- **Operaciones Primitivas:** Únicamente utiliza las consultas expuestas por la interfaz.
- **Abstracción Pura:** Garantiza que si la implementación interna del TDA cambia, las soluciones derivadas sigan funcionando intactas.



Vecinos Salientes



Grado Positivo / Negativo



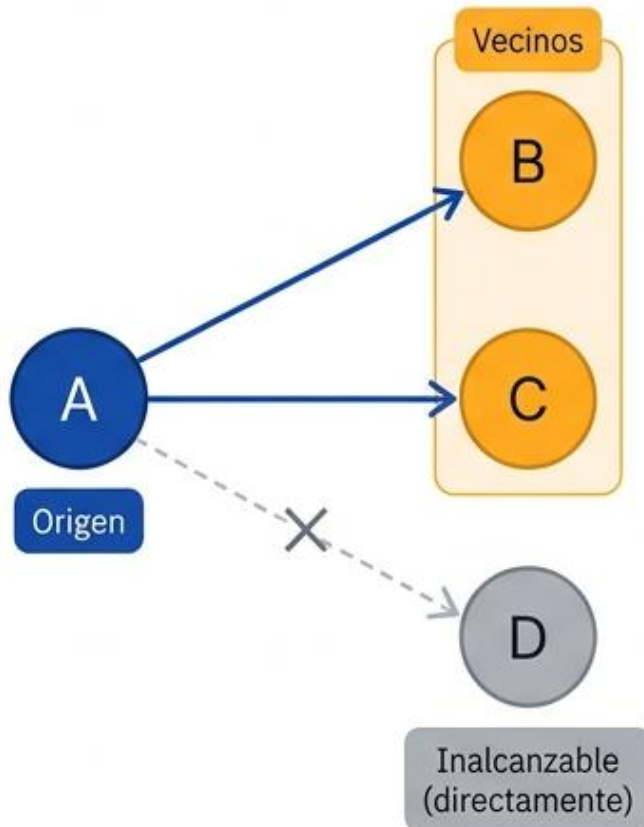
Detección de Aislados



Caminos y Alcanzables

Vecinos salientes: recorrer y consultar

El vecindario saliente está formado por todos los destinos directos a los que se puede llegar desde un vértice específico mediante una arista.



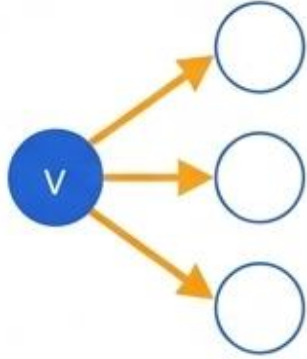
```
public ConjuntoTDA vecinosSalientes(GrafoTDA g, int origen) {  
    ConjuntoTDA vecinos = new ConjuntoLD();  
    vecinos.InicializarConjunto();  
    ConjuntoTDA vertices = g.Vertices();  
  
    while (!vertices.ConjuntoVacio()) {  
        int destino = vertices.Elegir();  
        vertices.Sacar(destino);  
        if (g.ExisteArista(origen, destino)) {  
            vecinos.Agregar(destino);  
        }  
    }  
    return vecinos;  
}
```

1. Estrategia iterativa:
Probar contra todos los nodos

2. Consultar conexión
directa: El orden de los
parámetros define el
sentido de la arista

3. Almacenar si es vecino

Grado positivo, grado negativo y vértices aislados

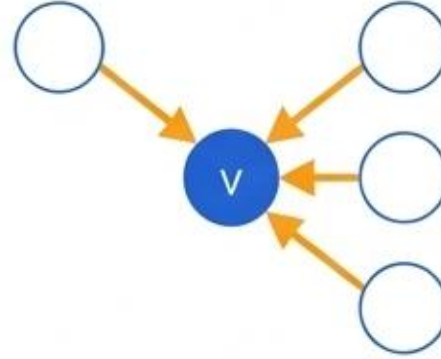


Grado Positivo

Cantidad de aristas salientes.

`ExisteArista(v, w)`
(v = base, w = iterador)

Ejemplo Aplicado: Una ciudad que es origen de múltiples vuelos directos a otros destinos.



Grado Negativo

Cantidad de aristas entrantes.

`ExisteArista(w, v)`
(w = iterador, v = base)

Ejemplo Aplicado: Una base de datos central que recibe información de múltiples microservicios dependientes.



Vértice Aislado

Grado Positivo == 0 Y
Grado Negativo == 0.

Ambas consultas lógicas retornan 0.

Ejemplo Aplicado: Un módulo de software obsoleto sin dependencias, o una ciudad bloqueada sin rutas viales activas.

Vecindario de un conjunto y complemento

Esquema Topológico



Panel de Estrategia Modular

1. Calcular vecindario:

```
ConjuntoTDA vecinos = vecinosConjunto(g, base);
```

2. Copiar grafo total:

Extraer `g.Vertices()` y realizar una copia exacta en memoria independiente.

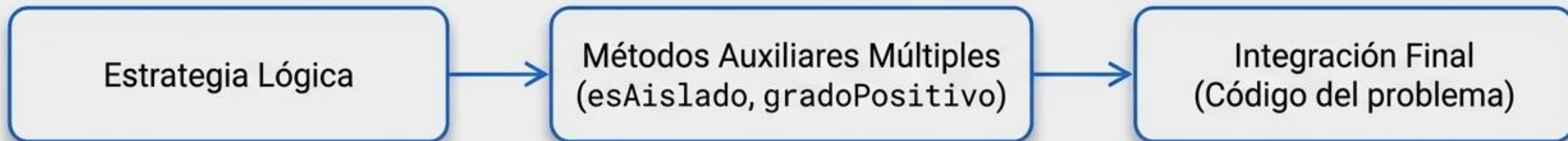
3. Filtrar / Restar:

Recorrer el conjunto vecinos y aplicar `copiaTotal.Sacar(v)` por cada uno.

Diseñar modularmente evita reescribir ciclos anidados complejos. Conceptualmente, el **complemento** es una simple resta de conjuntos: **Complemento = Total - Vecinos**

Modularidad y costo de las operaciones derivadas

La Forma Esperada de Resolución



🔵 **Por qué separar:** Permite aislar errores rápidamente, fomenta la reutilización de operaciones comunes y mantiene el código principal extremadamente declarativo y limpio.

Análisis de Costos (El Dilema de la Caja Negra)

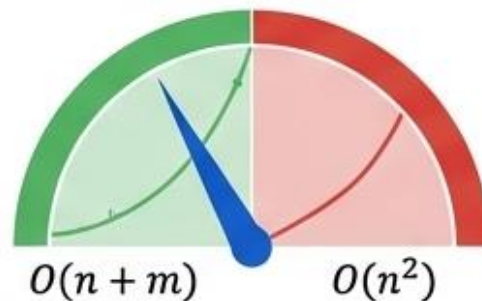
El Dilema

Si operamos estrictamente desde afuera del TDA sin que exponga sus listas de adyacencias, para obtener los vecinos de un nodo debemos iterar obligatoriamente todos los posibles destinos.

Costo $O(n^2)$

Por cada vértice iterado ($O(n)$), consultamos contra todos los vértices ($O(n)$) usando `ExisteArista()`.

El Factor Interno



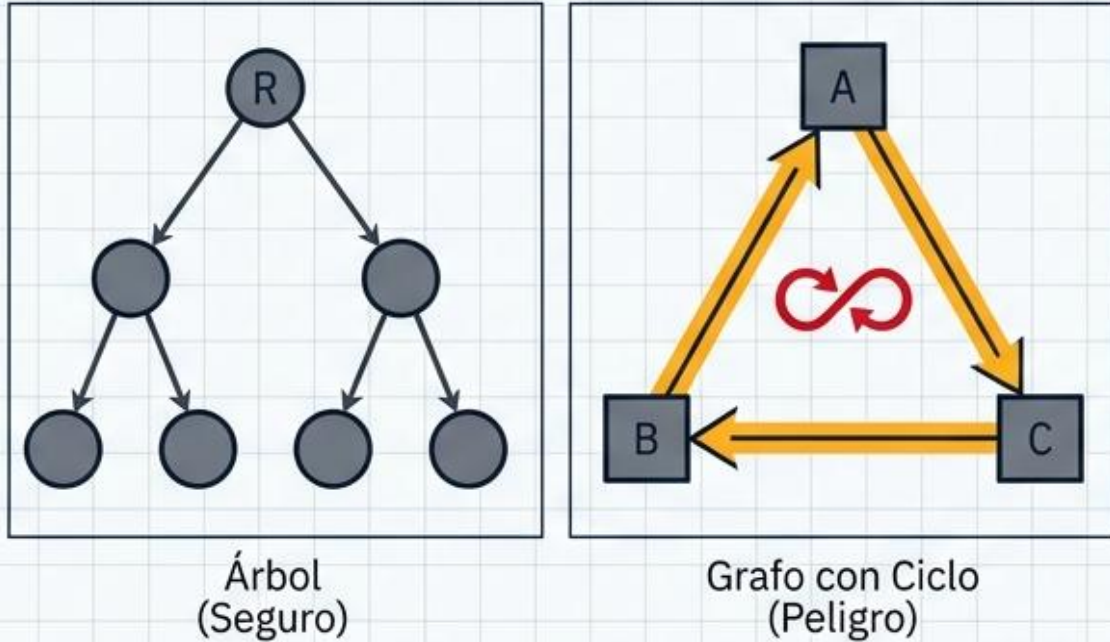
Si el TDA expusiera internamente la lista generada desde una Lista de Adyacencia, recorrer vecinos tomaría $O(n + m)$ (mucho más eficiente para redes dispersas).

Las abstracciones facilitan el diseño, pero la estructura interna domina el rendimiento físico.

Recorrer grafos: el problema de los ciclos

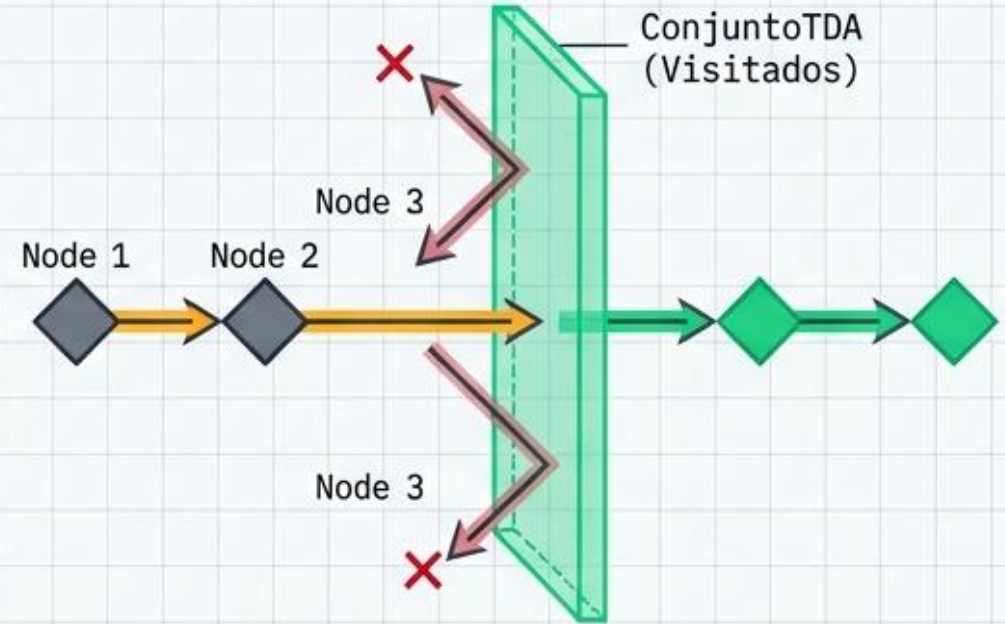
Recorrer un grafo no es igual a recorrer un árbol: existen caminos alternativos, retornos y ciclos infinitos.

El Problema: Repetición Indefinida



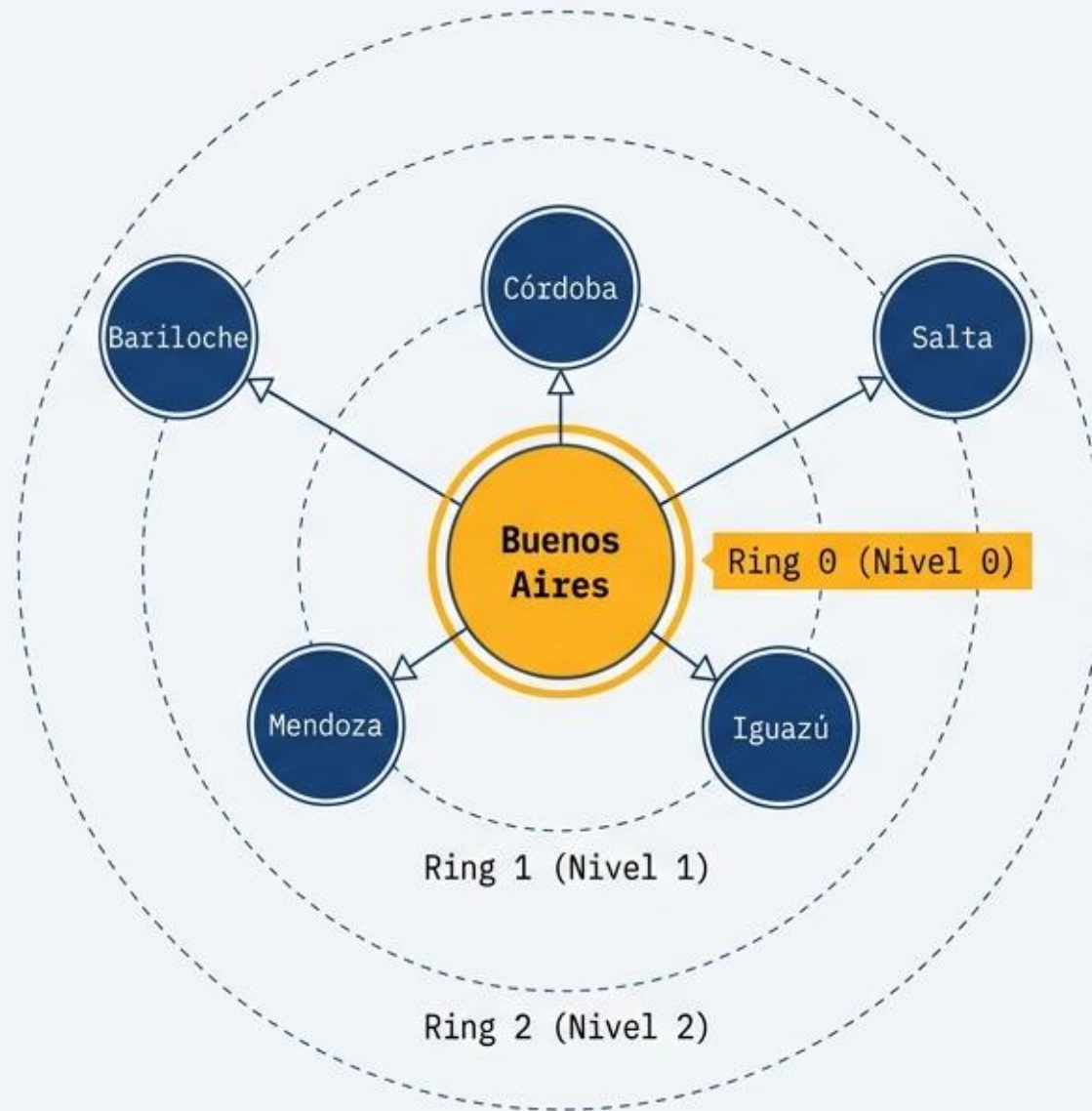
Sin una estructura de control, el algoritmo repite vértices indefinidamente. El camino **A -> B -> C -> A** colapsa la ejecución.

La Solución: Estructura de Control



El rol del **ConjuntoTDA**: Herramienta central para el control. Registra el historial de paso. Antes de avanzar a un vecino, se consulta el conjunto. Si ya fue visitado, el camino se descarta.

BFS: búsqueda en anchura



ColaTDA (FIFO)

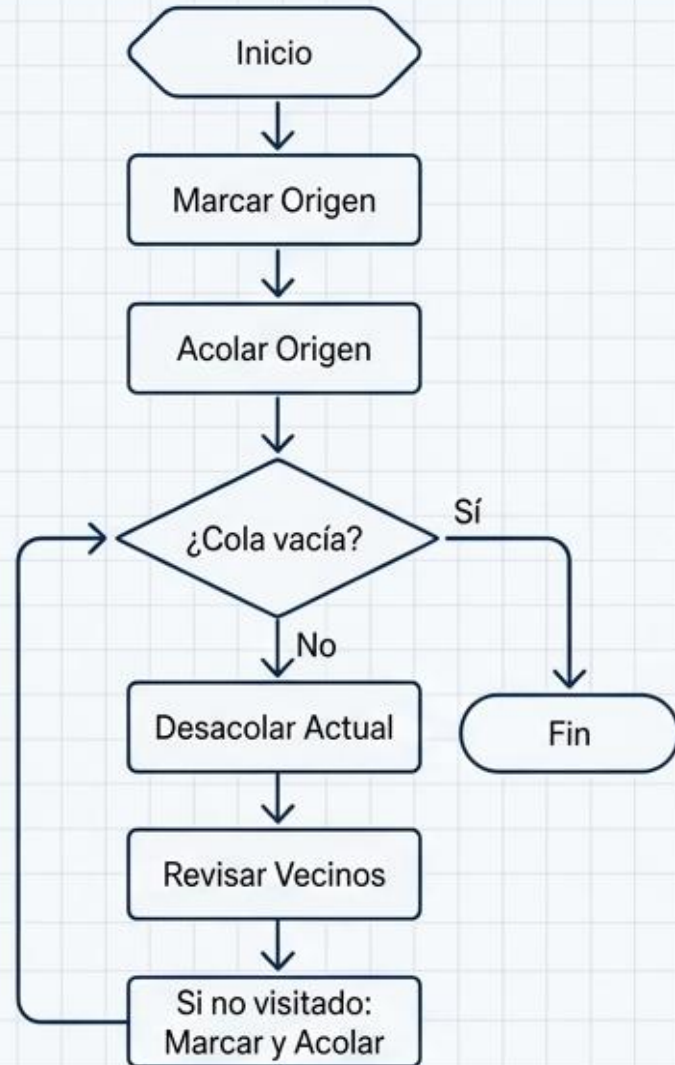
Estrategia: Breadth-First Search (BFS) es un recorrido expansivo por niveles. Descubre el mundo midiendo la proximidad.

Lógica: Visita primero el origen, luego todos sus vecinos directos, y recién después los vecinos de esos vecinos.

Estructura: Utiliza una ColaTDA porque procesa primero lo que descubrió primero, garantizando el orden FIFO (First In, First Out).

Caso de Uso Aplicado: Redes de vuelos (no ponderadas): Determinar la menor cantidad de escalas (saltos) para alcanzar cualquier ciudad destino.

Implementación conceptual de BFS



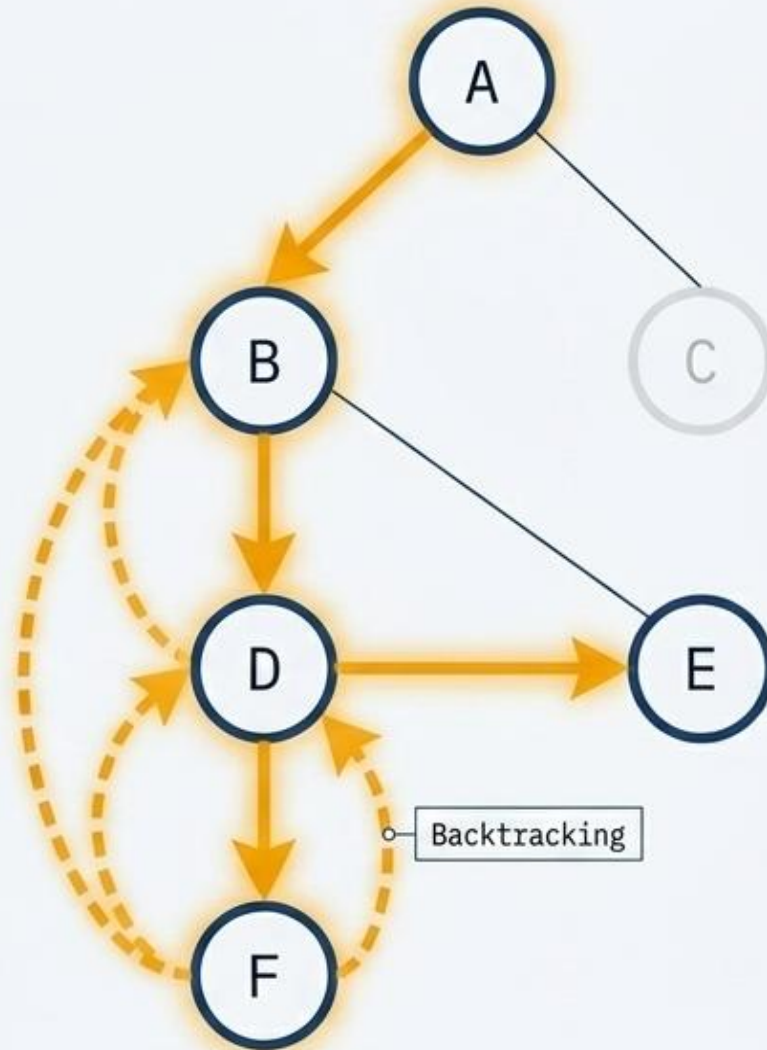
```
while (!pendientes.ColaVacia()) {  
    int actual = pendientes.Primerero();  
    pendientes.Desacolar();  
    ConjuntoTDA vertices = g.Vertices();  
  
    while (!vertices.ConjuntoVacio()) {  
        int vecino = vertices.Elegir();  
        vertices.Sacar(vecino);  
  
        if (g.ExisteArista(actual, vecino) && !visitados.Pertenece(vecino)) {  
            visitados.Agregar(vecino);  
            pendientes.Acolar(vecino);  
        }  
    }  
}
```



Detalle Crítico de Implementación:

Conviene marcar el vértice como visitado en el instante en que se descubre (antes de acolarlo), no cuando se procesa (al desacolarlo). Esto evita que un mismo vértice sea ingresado a la cola múltiples veces si es descubierto simultáneamente por diferentes caminos.

DFS: búsqueda en profundidad



PilaTDA /
Recursividad (LIFO)

Estrategia: Depth-First Search (DFS) es un recorrido exhaustivo. Avanza por un único camino hasta donde puede y luego retrocede.

Lógica: Sigue la ruta hasta un nodo sin salida, aplica backtracking (retroceso lógico) y explora las ramas pendientes.

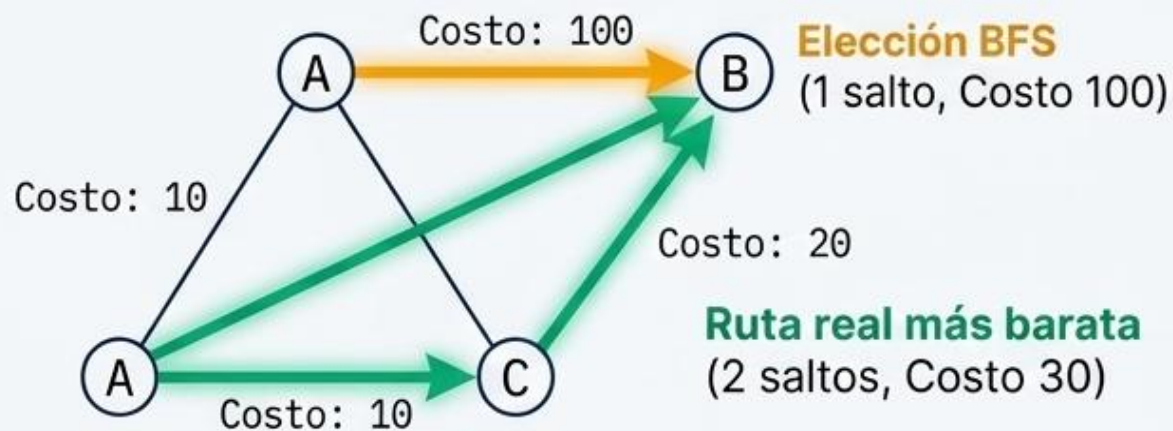
Estructura: Responde a la lógica LIFO (Last In, First Out). Se implementa de forma natural mediante recursividad (pila de llamadas del sistema) o explícitamente con una PilaTDA.

Caso de Uso Aplicado: Auditoría y Software: Análisis de dependencias profundas entre módulos de código, o seguimiento lineal de cadenas de eventos en procesos de auditoría.

BFS vs DFS: cuándo usar cada uno

Algoritmo	Estrategia	Estructura Auxiliar	Uso Típico
BFS	Explora cercano (por niveles)	Cola (FIFO)	Alcanzabilidad y menor cantidad de saltos en grafos no ponderados.
DFS	Explora profundo (backtracking)	Pila o Recursividad (LIFO)	Camino completo, cadenas de eventos y dependencias.

⚠ Limitación fundamental: Grafos Ponderados



Ni BFS ni DFS garantizan el menor costo cuando las aristas tienen pesos distintos.

En el ejemplo, BFS elegirá el camino directo A->B ignorando la ruta A->C->B que requiere más saltos pero tiene un costo real menor.

Para estos escenarios se requiere el Algoritmo de Dijkstra.

Cuando los pesos importan: del recorrido a la optimización

Por qué el camino más directo no siempre es el más conveniente

Los algoritmos BFS y DFS exploran la **topología (conexiones y saltos)**, pero **no garantizan la mejor ruta** si las aristas tienen costos distintos.



Costo (Ej: Tarifas de vuelos)



Tiempo (Ej: Logística y entregas)



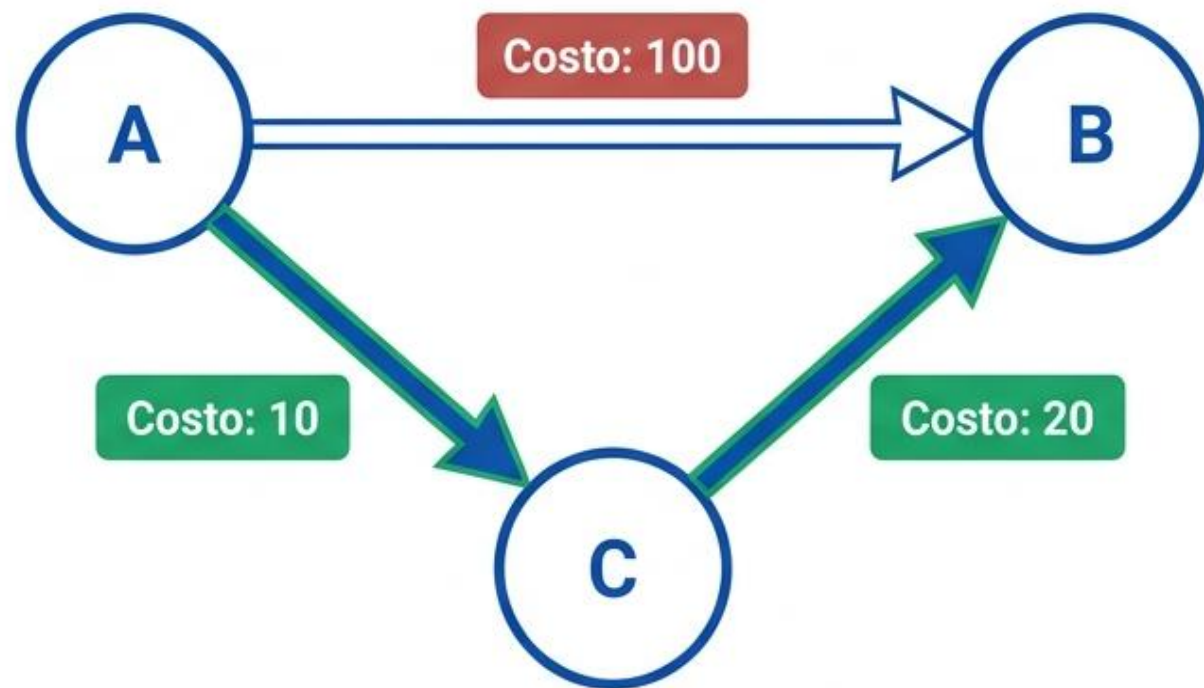
Distancia (Ej: Tramos de fibra óptica)



Riesgo (Ej: Auditoría de fraude)



Latencia (Ej: Ruteo de servidores)



→ **BFS** (Menor cantidad de saltos): **A → B** (1 salto). Costo total: **100**.

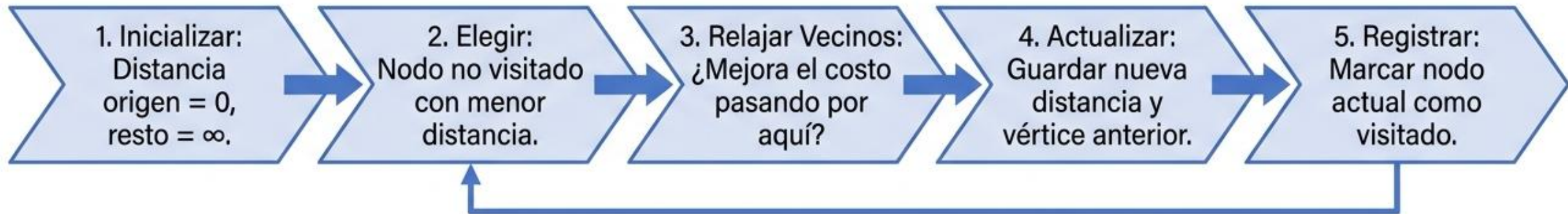
→ **Optimización** (Menor costo): **A → C → B** (2 saltos). Costo total: **30**.

El camino indirecto es más barato. BFS no puede detectar esta optimización.

Dijkstra: idea central

Algoritmo para caminos de menor costo en grafos ponderados (pesos no negativos)

Calcula el **menor costo acumulado** desde un vértice **origen** hacia todos los demás vértices del grafo.



Estado Inicial (Origen: Nodo A)

Vértice	Distancia	Anterior	Visitado
A	0	-1 (Ninguno)	Falso
B	∞	-1	Falso
C	∞	-1	Falso

El algoritmo opera actualizando estas tres estructuras paso a paso.

Relajación: mejorar una ruta conocida

El corazón de Dijkstra: minimizar el costo acumulado

Mecanics & Math

¿Qué es relajar una arista? Evaluar si pasar por el nodo actual (C) hacia un vecino (B) ofrece un costo acumulado menor que la ruta que ya conocíamos hacia B.

Math Block

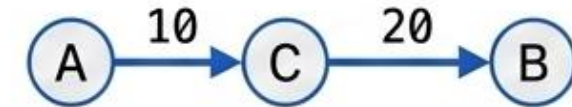
Distancia conocida a C: 10
Costo de la arista $C \rightarrow B$: 20
Nuevo Costo Propuesto = $10 + 20 = 30$

Evaluacion

Distancia anterior a B: 100
Como $30 < 100$, se actualiza la ruta.

Dijkstra no elige el camino más corto en cantidad de aristas, sino el menor costo acumulado.

Before & After Visualization



Estado Anterior

C : dist = 10, ant = A

B : dist = 100, ant = A



Relajación ($C \rightarrow B$)

Nuevo Estado

C : dist = 10, ant = A

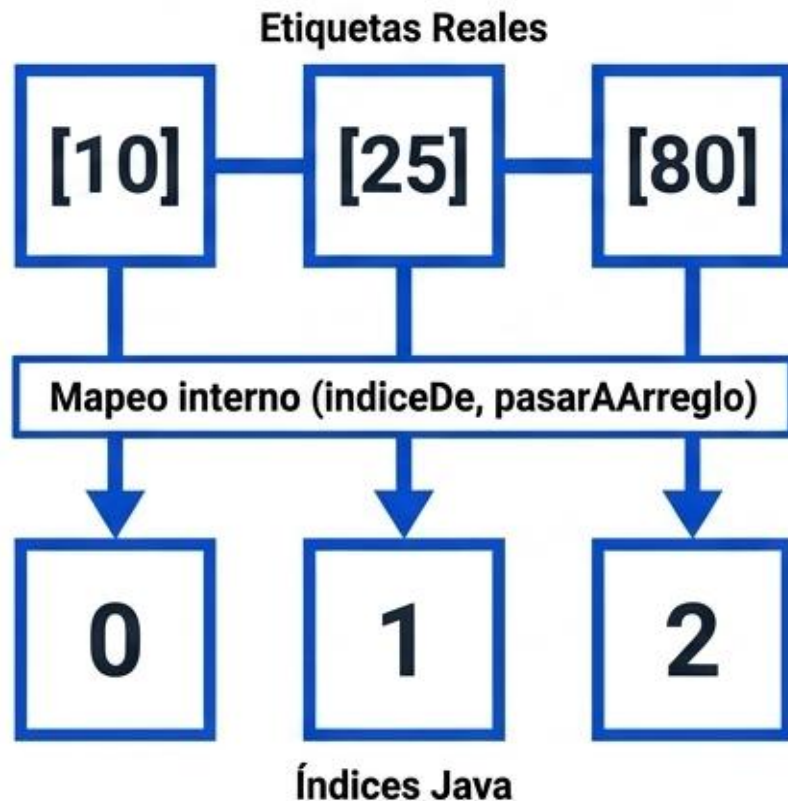
B : dist = 30, ant = C

Estructura implementable en Java

Traduciendo el modelo lógico a arreglos e índices internos

El problema del mapeo

En la realidad, los vértices no tienen IDs secuenciales desde 0 (Ej: Sucursales 10, 25, 80). Sin embargo, los arreglos en Java usan índices continuos 0, 1, 2...



Implementación & Auxiliares

	Arreglos Conceptuales Paralelos		
	0	1	2
int[] vertices			
int[] distancia			
int[] anterior			
boolean[] visitado			

```
// Buscar el vértice no visitado con menor distancia
public int menorNoVisitado(int[] distancia, boolean[] visitado, int n) {
    int posMenor = -1;
    int menor = INFINITO;
    for (int i = 0; i < n; i++) {
        if (!visitado[i] && distancia[i] < menor) {
            menor = distancia[i];
            posMenor = i;
        }
    }
    return posMenor;
}
```

La modularidad es clave: funciones auxiliares abstraen la complejidad de los arreglos geométricos.

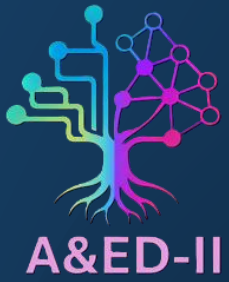
Elegir el algoritmo correcto y justificar el costo

Síntesis comparativa para la toma de decisiones algorítmicas

Pregunta del Negocio / Problema	Estrategia Algorítmica	Costo Estimado
¿Existe conexión directa?	Consulta de TDA (ExisteArista)	$O(1)$ a $O(n)$ (Matriz vs Lista)
¿El destino es alcanzable?	Recorrido BFS o DFS (con visitados)	$O(n^2)$ (TDA base) o $O(n + m)$ (Lista)
¿Camino con menor cant. de saltos? (Sin pesos)	Recorrido BFS	$O(n^2)$ (TDA base) o $O(n + m)$ (Lista)
¿Ruta de menor costo acumulado? (Pesos positivos)	Dijkstra	$O(n^2)$ (Didáctico con arreglos)

n = vértices, m = aristas. GrafoTDA base sin acceso directo a vecinos penaliza a $O(n^2)$ por revisión exhaustiva.

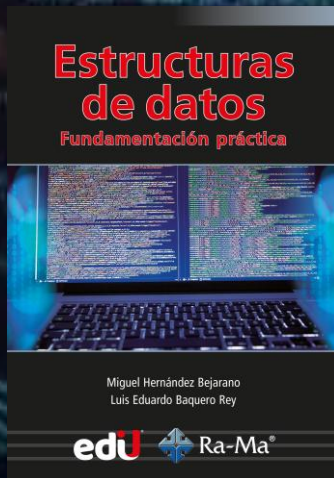
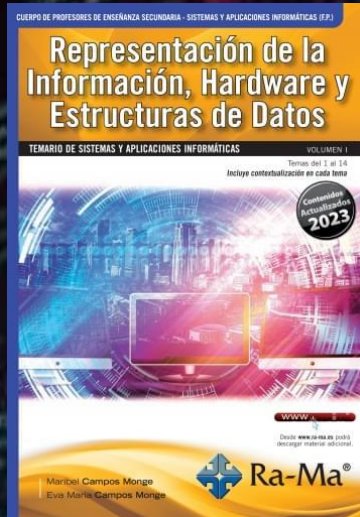




Ejercitación

Los ejercicios tienen como objetivo fortalecer el dominio de los temas centrales de la materia: árboles, árboles binarios de búsqueda, árboles balanceados, grafos, recorridos, estructuras dinámicas, nodos, listas enlazadas, pilas, colas, modularidad y análisis de costos.

La intención no es memorizar código, sino comprender cómo se analiza un problema, qué estructura conviene utilizar, qué métodos auxiliares pueden simplificar la solución y cómo justificar el costo temporal de una estrategia. En todos los casos, se espera una respuesta clara, ordenada y técnicamente fundamentada. Cuando corresponda, puede acompañarse la explicación con pseudocódigo, fragmentos de Java o diagramas simples.



BIBLIOGRAFIA



Cuadrado Estrebou, María Fernanda Trutner, Guillermo Hernán. Revisión: J.Tempio.
Programación II. Apuntes de cátedra.